

C / C++ Theory — IBM OA Notes

Definition → Example → Important Difference → Exam Trap

1. Pointer kya hota hai?

Pointer ek variable hota hai jo kisi doosre variable ka memory address store karta hai.

```
int x = 10;
int *p = &x;
```

Yahan:

- `x` → value 10
- `&x` → x ka address
- `p` → x ka address store karta hai
- `*p` → us address par stored value, yani 10

Important operators

- `&` → address-of operator
- `*` → dereference operator

```
cout << p;    // address
cout << *p;   // value
```

2. Dangling Pointer 🚬

A dangling pointer woh pointer hai jo kisi aise memory location ko point kar raha hai jo already deallocated/free ho chuki hai.

Example:

```
int *p = new int(10);
delete p;
// p is now dangling
```

`delete p` ke baad memory free ho gayi, lekin p abhi bhi purane address ko contain karta hai.

Avoid:

```
delete p;
p = nullptr;
```

Exam definition: A dangling pointer points to memory that has already been freed or is no longer valid.

3. Wild Pointer 🚬

A wild pointer ek aisa pointer hai jo initialize nahi kiya gaya.

```
int *p;
```

p ke andar random/indeterminate address ho sakta hai.

Difference

Dangling	Wild
Memory valid nahi rahi	Pointer initialize hi nahi hua
Usually delete/lifetime end ke baad	Declaration ke baad
Points to invalid/deallocated memory	Unknown address

Safe practice:

```
int *p = nullptr;
```

4. Pointer vs Reference 🚬

Pointer

```
int x = 10;
int *p = &x;
```

- Address store karta hai
- `nullptr` ho sakta hai
- Baad mein doosre variable ko point kar sakta hai
- `*` se dereference hota hai

Reference

```
int x = 10;
int &r = x;
```

- Existing variable ka alias hai
- Normally initialize karna mandatory hai
- `nullptr` nahi hota

- Reference ko baad mein kisi doosre variable se rebind nahi kar sakte
- Use karte waqt dereference `*` nahi lagate

```
r = 20; // means x becomes 20
```

याद रखो — **Pointer = address holder, Reference = another name/alias**

5. NULL vs nullptr

Old C/C++ style:

```
int *p = NULL;
```

Modern C++:

```
int *p = nullptr;
```

`nullptr` specifically null pointer literal hai. Iska type `std::nullptr_t` hai.

Why better? NULL generally 0 ya 0L ke form mein define ho sakta hai, jiski wajah se function overloading mein ambiguity ho sakti hai. `nullptr` specifically pointer ke liye hai.

OA answer: `nullptr` is the type-safe null pointer introduced in C++11.

6. Stack vs Heap

Stack

Generally stores: Local variables, Function call information, Parameters, Stack frames

```
void fun() {
    int x = 10;
}
```

x automatic storage duration wala local variable hai.

Heap / Free Store

Dynamic allocation ke liye use hota hai.

```
int *p = new int(10);
```

Memory manually manage karni pad sakti hai: `delete p;`

Stack	Heap
Automatic storage commonly	Dynamic allocation
Fast allocation/deallocation generally	Generally more overhead
Limited size	Usually larger
Local variables/function frames	Dynamically allocated objects
Automatically managed	new/delete or smart pointers

△ Exact implementation details compiler/OS par depend kar sakti hain, so OA mein basic conceptual answer yaad rakho.

7. malloc() vs calloc()

Both C-style dynamic memory allocation functions hain.

malloc

```
int *p = (int*)malloc(5 * sizeof(int));
```

Memory allocate karta hai but initialization nahi karta.

calloc

```
int *p = (int*)calloc(5, sizeof(int));
```

Memory allocate karta hai aur allocated bytes ko zero-initialize karta hai.

malloc	calloc
1 argument	2 arguments
malloc(total_bytes)	calloc(number, size)
Memory uninitialized	Memory zero-initialized
C function	C function

C++ mein generally new, containers aur smart pointers preferred hote hain.

8. malloc() vs new

malloc

```
int *p = (int*)malloc(sizeof(int));
```

- C-style function
- Raw memory allocate karta hai
- Constructor call nahi karta
- `free()` se release karna hota hai

new

```
int *p = new int(10);
```

- C++ operator
- Object create karta hai
- Constructor call karta hai
- `delete` se release karte hain

malloc() → free() | new → delete — Don't mix them.

```
✗ int *p = new int; free(p);
✗ int *p = (int*)malloc(sizeof(int)); delete p;
```

9. Memory Leak 🔥

Memory leak tab hota hai jab dynamically allocated memory ko release nahi kiya jata aur program us memory ko access karne ka useful reference bhi lose kar deta hai.

```
int *p = new int(10);
// forgot delete
```

Agar aisa repeatedly hota rahe, program unnecessary memory consume karta rahega.
Avoid: `delete p;` — Modern C++ mein RAIL / smart pointers use karna better approach hai.

10. Shallow Copy vs Deep Copy 🔥🔥

Suppose class ke andar pointer hai:

```
class A {
public:
    int *p;
};
```

Shallow Copy

Pointer ka address copy hota hai. So: Object A → address 1000, Object B → address 1000. Dono same memory ko point kar rahe hain.
Problem: ek object memory delete kare, toh doosra dangling pointer bana sakta hai.

Deep Copy

Actual data ki separate copy banayi jaati hai. Object A → 1000 → 50, Object B → 2000 → 50. Dono independent memory use karte hain.

Remember: Shallow copy → same resource | Deep copy → separate resource

11. Copy Constructor 🔥

Copy constructor ek existing object se new object initialize karta hai.

```
class A {
public:
    A(const A& obj) {
        // copy
    }
};

A a;
A b = a; // b ko a se initialize kiya ja raha hai
```

Important: Copy constructor ka parameter generally reference hota hai: `const A&` . Agar value se pass karoge, copy constructor ko call karne ke liye again copy chahiye → problematic/infinite recursion.

12. Constructor vs Destructor 🔥

Constructor

Object create hone par automatically call hota hai. Purpose: Initialization

```
class A {
public:
    A() {
        cout << "Constructor";
    }
};
```

Destructor

Object destroy hone par automatically call hota hai. Purpose: Cleanup, Resources release karna

```
~A() {
    cout << "Destructor";
}
```

Constructor	Destructor
Object initialization	Object cleanup
Same class name	~ClassName()
Multiple constructors possible	Only one destructor
Can be overloaded	Cannot be overloaded
No return type	No return type

13. Virtual Function 🔥🔥

Virtual function ka main purpose hai runtime polymorphism.

```
class Parent {
public:
    virtual void show() {
        cout << "Parent";
    }
};

class Child : public Parent {
public:
    void show() override {
        cout << "Child";
    }
};

Parent *p = new Child();
p->show();
// Output: Child
```

Because function call runtime par actual object type ke according resolve hota hai.

Important keyword: `virtual`

14. Pure Virtual Function 🔥

A pure virtual function ki declaration mein `= 0` hota hai.

```
class Shape {
public:
    virtual void draw() = 0;
};
```

Aisi class ko **abstract class** kehte hain. You cannot normally create its object: `Shape s; // ✖`

Derived class ko pure virtual function implement karna padega, otherwise derived class bhi abstract rahegi.

Remember: virtual void draw() = 0; → Pure virtual function

15. Function Overloading 🔥

Same function name, but different parameter list.

```
int add(int a, int b);
double add(double a, double b);
```

This is **compile-time polymorphism**. Parameters can differ by: Number, Type, Order.

Return type alone se overloading nahi hoti. ✖

```
int add(int a);
double add(int a);
```

16. Function Overriding 🔥

Derived class parent ke function ka new implementation provide karti hai.

```
class Parent {
public:
    virtual void show() {}
};

class Child : public Parent {
public:
    void show() override {}
};
```

This is associated with **runtime polymorphism**, particularly when using virtual dispatch.

Overloading	Overriding
Same class commonly	Parent-child relationship
Different parameters	Same function signature/compatible override
Compile time	Runtime with virtual dispatch

17. Compile-time vs Runtime Polymorphism

Compile-time — Compiler decide karta hai. Examples: Function overloading, Operator overloading

Runtime — Program execution ke time decide hota hai. Main example: Virtual function overriding

Shortcut: Overloading → Compile time | Overriding + virtual → Runtime

18. static Keyword 🔥

`static` ka meaning context ke according change hota hai.

Static local variable

```
void fun() {
    static int x = 0;
    x++;
}
```

x function calls ke beech apni value retain karega. Normal local variable har function call par new lifetime start karta hai.

Static class member

```
class A {
public:
    static int count;
};
```

Static data member class ke saare objects ke beech shared hota hai. You don't need a separate copy for every object.

Static function

```
static void fun();
```

Class ke static member function ke paas normally `this` pointer nahi hota.

19. const Keyword 🔥

`const` means value/object ko modify nahi karna.

Const variable

```
const int x = 10;
x = 20; // x
```

Pointer cases — Very important

```
const int *p;           // pointer can change, value cannot be changed through p
int *const p = &x;      // pointer cannot change, value can be modified through p
const int *const p = &x; // neither pointer nor pointed value can be changed through p
```

Ye OA mein favourite trap hai.

20. this Pointer 🔥

`this` pointer current object ko point karta hai.

```
class Student {
    int age;

public:
    void setAge(int age) {
        this->age = age;
    }
};
```

Left side `this->age` → current object's age. Right side `age` → function parameter.

Important:

- this pointer non-static member functions mein available hota hai
- Current object ko refer karta hai
- Static member function mein this pointer nahi hota

21. Pass by Value vs Pass by Reference 🔥

Pass by Value

```
void change(int x) {
    x = 20;
}
```

Original variable change nahi hota.

Pass by Reference

```
void change(int &x) {
    x = 20;
}
```

Original variable change ho jayega.

```
int a = 10;
change(a);
// Pass by value: a = 10
// Pass by reference: a = 20
```

22. Pointer Arithmetic 🗝

Pointers ke saath arithmetic possible hai, especially arrays mein.

```
int arr[] = {10,20,30,40};
int *p = arr;

*(p + 1)    // → 20
```

Why? Because pointer arithmetic element size ke according move karta hai. If p is int*, p + 1 means next int, not necessarily just one byte (i.e., i × sizeof(int)).

Index	Value	Address
a[0]	12	0xA0
a[1]	7	0xA4
a[2]	-3	0xA8
a[3]	40	0xAC
a[4]	9	0xB0
a[5]	21	0xB4
a[6]	5	0xB8
a[7]	16	0xBC

Important: for `arr[i]`, equivalent form is `*(arr + i)`

23. Segmentation Fault 🗝

Segmentation fault occurs when a program tries to access memory that it is not allowed to access.

Examples

Invalid pointer

```
int *p = nullptr;
cout << *p;
```

Out-of-bounds access

```
int arr[3];
cout << arr[10];
```

Accessing freed memory

```
int *p = new int(10);
delete p;
cout << *p;
```

This is undefined behavior and may result in a segmentation fault.

Segmentation fault = invalid memory access.

24. struct vs class 🗝

C++ mein dono user-defined types hain. Main difference: members default access aur inheritance default.

```
struct A {
    int x;    // public
};

class A {
    int x;    // private
};
```

Inheritance ka default bhi different hota hai:

```
struct Derived : Base    // → public inheritance by default
class Derived : Base     // → private inheritance by default
```

25. Vector vs Array 🗝

Array

Fixed-size contiguous collection.

```
int arr[5]; // Size fixed
```

Vector

Dynamic-size container.

```
vector<int> v;  
v.push_back(10);
```

Size dynamically grow/shrink kar sakta hai.

Vector advantages: Dynamic size, STL algorithms ke saath easy, push_back(), size(), Iterators

vector internally dynamically allocated storage use karta hai and its elements are stored contiguously.

26. map vs unordered_map 🌱🌱

map

Usually implemented as a balanced search tree.

- Keys sorted
- Search: $O(\log n)$
- Insert: $O(\log n)$
- Delete: $O(\log n)$

unordered_map

Hash table based.

- Keys ordered nahi hote
- Average search: $O(1)$
- Average insert: $O(1)$
- Average delete: $O(1)$

Worst case can degrade to $O(n)$.

map → sorted → $O(\log n)$ | unordered_map → no order → average $O(1)$

27. Stack vs Queue 🌱

Stack — LIFO (Last In First Out)

```
Push: 1 2 3  
Pop → 3
```

```
stack<int> s; // push(), pop(), top()
```

Queue — FIFO (First In First Out)

```
1 2 3  
pop → 1
```

```
queue<int> q; // push(), pop(), front(), back()
```

Stack → LIFO | Queue → FIFO

28. Iterator 🌱

Iterator ek object hai jo STL container ke elements ko traverse/access karne ke liye use hota hai.

```
vector<int> v = {10,20,30};  
vector<int>::iterator it;  
  
for(it = v.begin(); it != v.end(); it++) {  
    cout << *it;  
}
```

Modern C++ mein:

```
for(auto it = v.begin(); it != v.end(); ++it)
```

Important functions: begin() → first element ko point karta hai. end() → last element ke baad position ko represent karta hai. end() ko dereference nahi karna chahiye.

29. Important STL Time Complexities 🌱🌱

OA ke liye ye table yaad kar lo:

Operation	Complexity
vector[i]	O(1)
vector.push_back()	O(1) amortized
vector.pop_back()	O(1)
vector insert beginning	O(n)
vector erase beginning	O(n)
map.find()	O(log n)
map.insert()	O(log n)
map.erase()	O(log n)
unordered_map.find()	O(1) average
unordered_map.insert()	O(1) average
set.find()	O(log n)
unordered_set.find()	O(1) average
stack.push()	O(1)
stack.pop()	O(1)
stack.top()	O(1)
queue.push()	O(1)
queue.pop()	O(1)
queue.front()	O(1)
priority_queue.push()	O(log n)
priority_queue.pop()	O(log n)
priority_queue.top()	O(1)
sort()	O(n log n)

⚠ unordered_map / unordered_set ka average O(1) hai, guaranteed worst-case O(1) nahi.

30. Inheritance + Virtual Function — MUST UNDERSTAND 🔥🔥

Ye output questions mein bahut important hai.

```
class A {
public:
    virtual void show() {
        cout << "A";
    }
};

class B : public A {
public:
    void show() override {
        cout << "B";
    }
};

A *ptr = new B();
ptr->show();
// Output: B
```

Why? ptr ka pointer type: A*, but actual object: B. Because show() is virtual, runtime polymorphism occurs. So actual object's overridden function executes.

What if virtual hata dein?

```
class A {
public:
    void show() {
        cout << "A";
    }
};

A *ptr = new B();
ptr->show();
// will resolve based on the pointer/reference's static type: A
```

Golden rule: virtual → runtime dispatch | non-virtual → static/compile-time binding